

TheKnife

Manuale Utente

Università degli Studi dell'Insubria – Laurea Triennale in Informatica

Erik Mirashaj, 760453

Kabuka Dan Mumanga, 757708

Igor Gorchynskyi, 757184

Lorenzo Mujeci, 757597

Data: 29/05/2026

Versione Documento: 1.4

Indice

1. Introduzione tecnica
2. Installazione
 - 2.1 Requisiti di sistema
 - 2.2 Setup ambiente
 - 2.3 Installazione programma
3. Esecuzione ed uso
 - 3.1 Lancio del programma
 - 3.2 Uso tecnico delle funzionalità
4. Data set di test
5. Progettazione tecnica
 - 5.1 Architettura client/server
 - 5.2 Struttura a package
 - 5.3 Scelte progettuali (UML)
 - 5.4 Documentazione database (ER)
 - 5.5 Strutture dati utilizzate
 - 5.6 Scelte algoritmiche
 - 5.7 Pattern utilizzati
 - 5.8 JavaDoc
6. Bibliografia

Introduzione tecnica

TheKnife è un sistema software client/server sviluppato in linguaggio Java, progettato per la gestione, esplorazione e prenotazione di servizi ristorativi. Il sistema si articola su tre componenti principali:

- **Client desktop JavaFX** (theknife.client): gestisce lo strato di presentazione e l'interazione utente attraverso un'interfaccia grafica moderna.
- **Server TCP** (theknife.server): nucleo centrale che gestisce le richieste dei client, l'accesso ai dati e le regole applicative.
- **Database PostgreSQL**: garantisce la persistenza dei dati attraverso uno schema relazionale definito.

Il progetto utilizza Apache Maven per il build management, il packaging e la generazione automatizzata della documentazione tecnica.

Installazione

Requisiti di sistema

Per il corretto funzionamento del sistema sono necessari i seguenti prerequisiti:

- **JDK:** Versione 26.
- **Maven:** Versione 3.9 o superiore.
- **PostgreSQL:** Versione 14 o superiore.
- **Rete:** Una porta TCP libera (default: 5000) per la comunicazione tra i moduli.

Le dipendenze principali gestite nel file pom.xml includono JavaFX (controls e fxml), PostgreSQL JDBC per la connettività DB, BCrypt per la sicurezza delle password, AtlantaFX per lo stile UI e HikariCP per il pooling delle connessioni.

Setup ambiente

1. Installare il Java Development Kit (JDK 26) e Maven.
2. Installare e configurare l'engine PostgreSQL.
3. Creare un database applicativo usando il comando
“createdb -U *nomeutente* dbtk”
4. Ripristinare il database tramite il file dump:
“pg_restore -U *nomeutente* -d dbtk Backup.sql”
5. Verificare che le credenziali nel codice del server (ServerTK)
corrispondano a quelle configurate nel DB.

Installazione programma

La build completa del progetto viene eseguita tramite il comando:
mvn clean package

Gli artefatti prodotti includono i file JAR per client e server posizionati nella cartella bin/, mentre la documentazione JavaDoc viene generata in doc/javadoc.

Esecuzione ed uso

Lancio del programma

Per avviare correttamente il sistema, bisogna semplicemente fare doppio clic sul file “run-server.bat/sh” e successivamente nel file “run-client.bat/sh” a seconda del sistema operativo che si sta utilizzando.

Uso tecnico delle funzionalità

- **Comunicazione:** Il client inizializza una connessione socket verso il server (default localhost:5000) tramite ServerConnection.
- **Protocollo:** Le interazioni sono incapsulate in oggetti Request e Response condivisi tra le parti.
- **Concorrenza:** Il server adotta un modello multithreading, assegnando un ClientHandler dedicato a ogni nuova connessione.
- **Data Access:** L'accesso al database è mediato da un layer DAO (Data Access Object).

Data set di test

Sono stati utilizzati diversi strumenti per la gestione, la creazione e il test dei dati:

- **Schema:** File sql/create_db.sql per l'inizializzazione delle tabelle.
- **Script di supporto:** Script Python (generate_reviews.py, csv_to_sql.py) per la generazione di recensioni e la conversione di dati CSV.

Progettazione tecnica

Architettura client/server

Il sistema segue un'architettura a tre livelli logici:

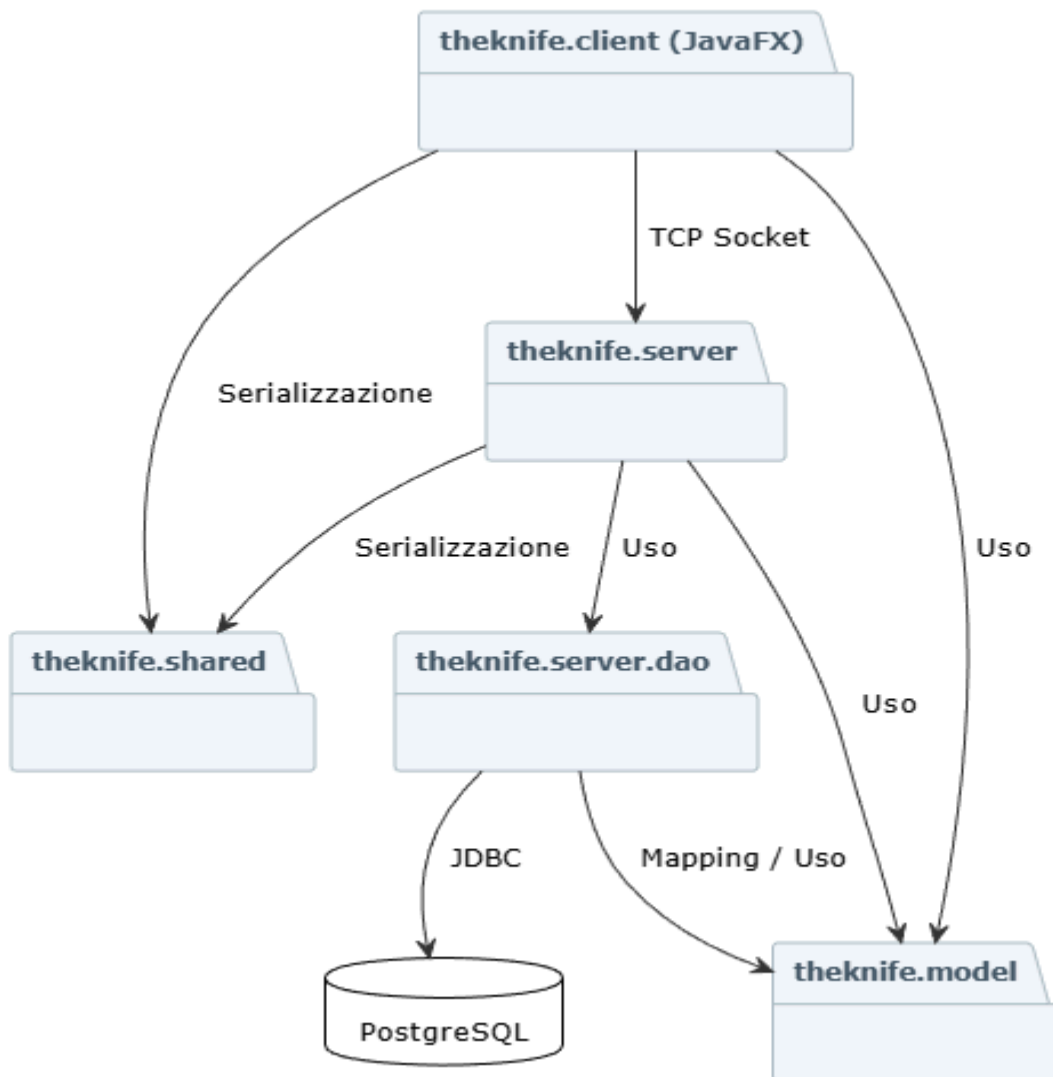
1. **Presentation Layer:** Interfaccia JavaFX e relativi controller.
2. **Application Layer:** Gestione della logica di business e del protocollo di comunicazione nel server.
3. **Data Access Layer:** Implementazione dei pattern DAO e pooling JDBC tramite HikariCP per PostgreSQL.

Struttura a package

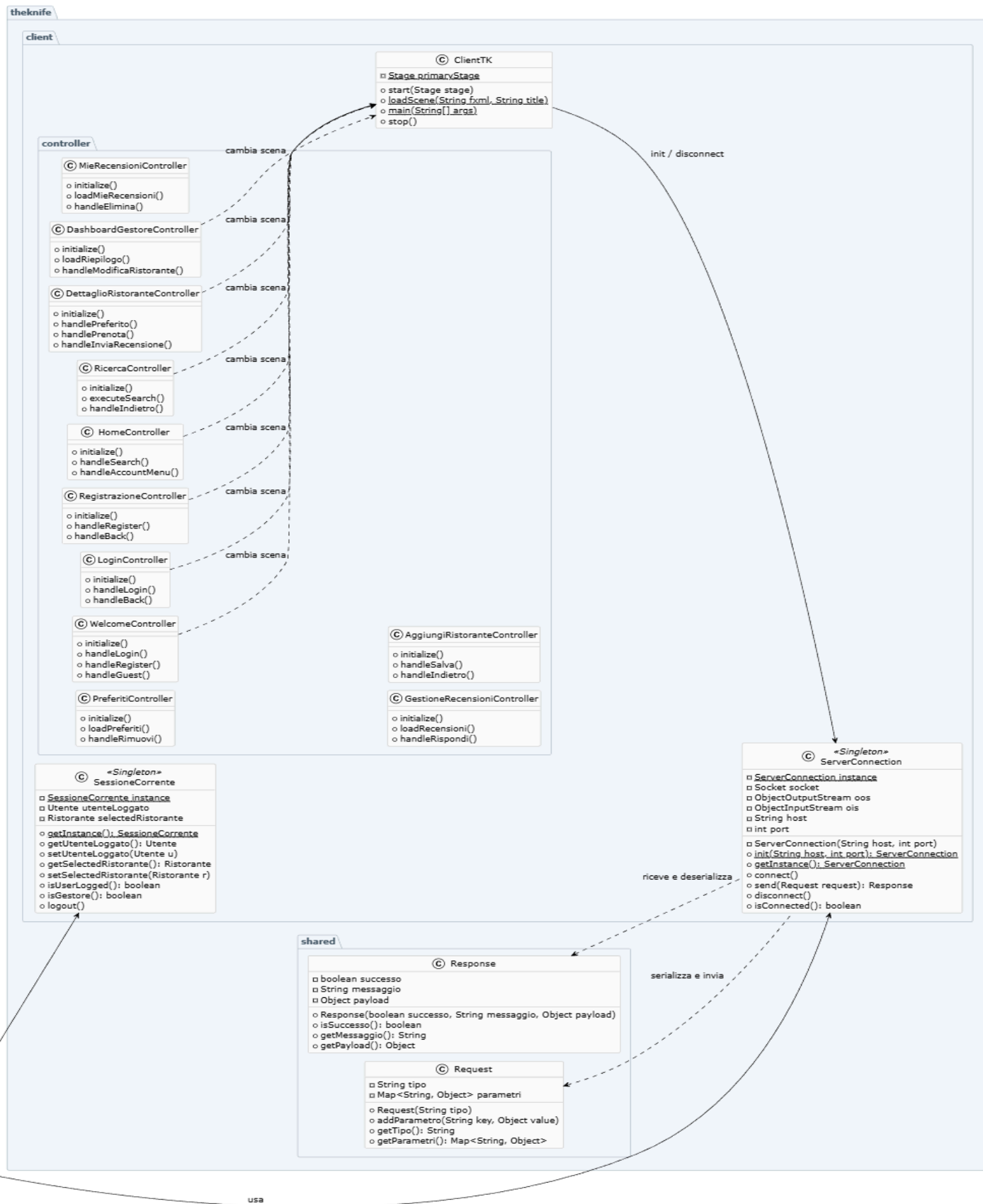
Package	Descrizione
theknife.client	Bootstrap del client e gestione sessione.
theknife.client.controller	Controller per la logica dell'interfaccia FXML.
theknife.model	Entità di dominio (Utente, Ristorante, ecc.).
theknife.server	Nucleo del server e gestione connessioni.
theknife.server.dao	Interfacce e implementazioni per l'accesso ai dati.
theknife.shared	Oggetti DTO e protocollo di comunicazione.

Scelte progettuali (UML)

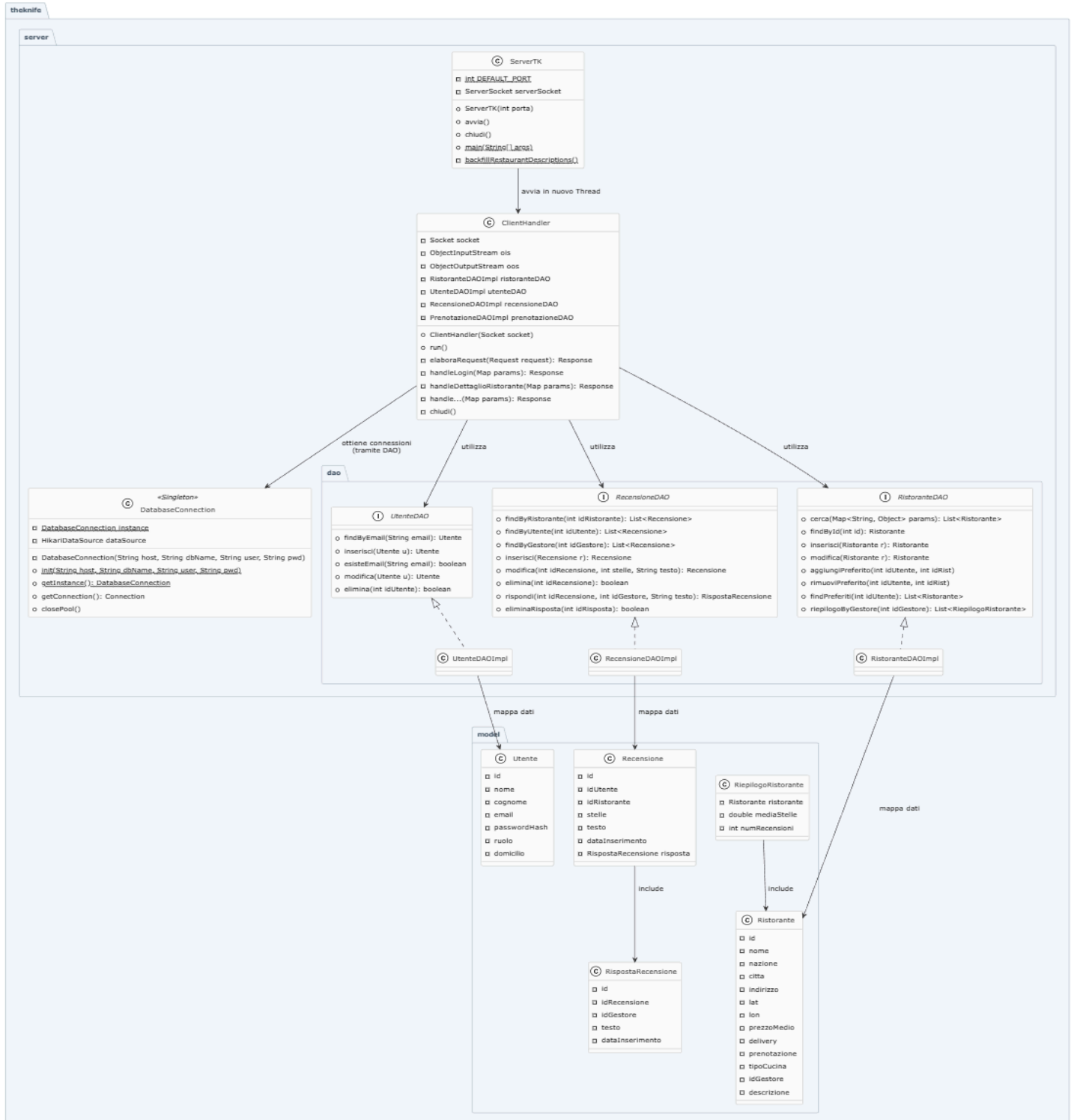
ARCHITETTURA GENERALE (SOLO PACKAGE E DIPENDENZE PRINCIPALI)



LATO CLIENT (ClientTK, ServerConnection, SessioneCorrente, Controller, Request/Response)



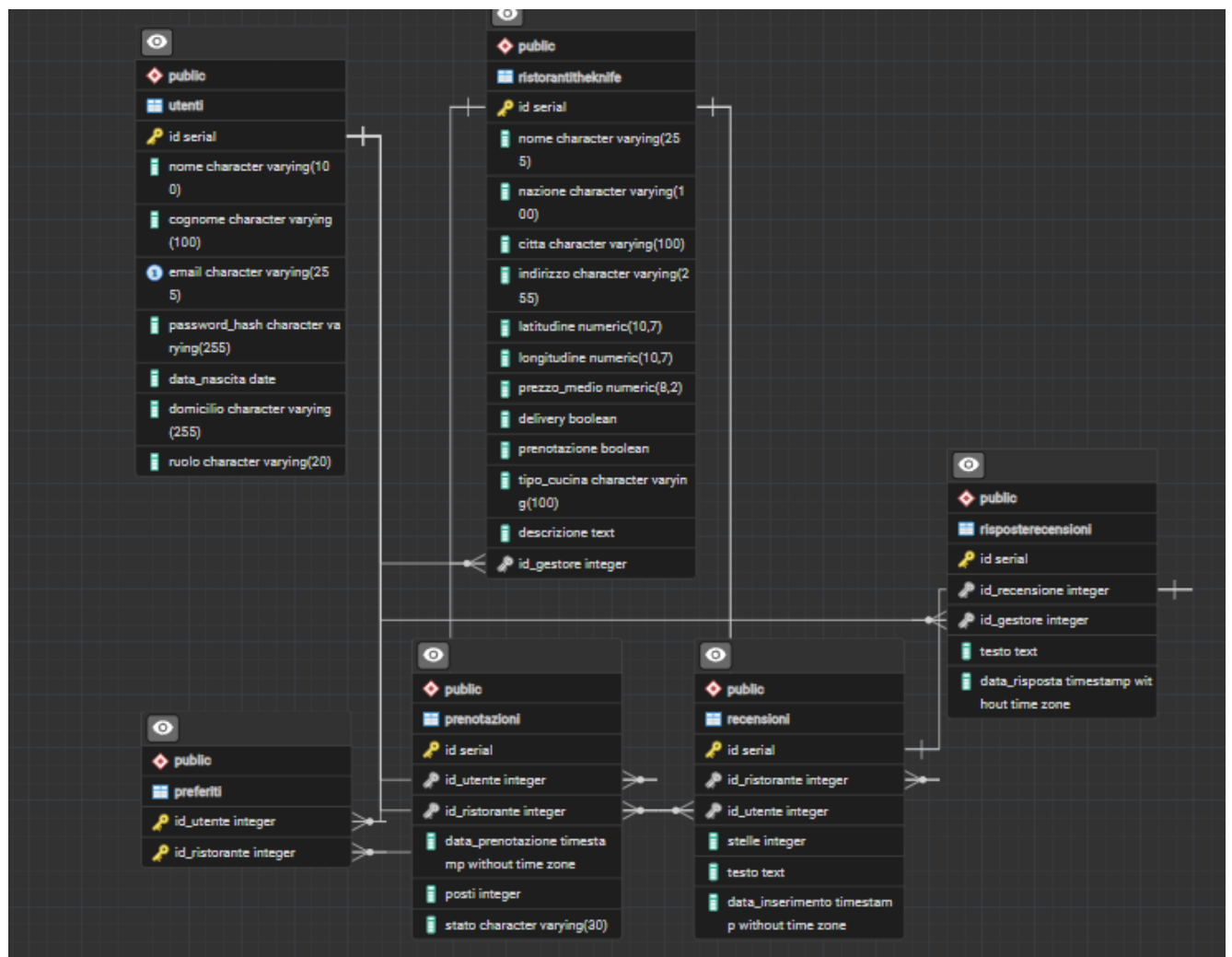
LATO SERVER (ServerTK, ClientHandler, tutti i DAO interfaces + impl, DatabaseConnection, Model)



Documentazione database (ER)

Lo schema relazionale gestisce le seguenti entità:

- **Utenti:** Gestisce ruoli ('cliente', 'gestore') e credenziali.
- **RistorantiTheKnife:** Informazioni sui ristoranti.
- **Recensioni/Risposte:** Gestisce i feedback dei clienti e le risposte dei gestori.
- **Preferiti/Prenotazioni:** Relazioni tra utenti e locali con attributi temporali (LocalDateTime).



Pattern utilizzati

- **Singleton:** Applicato a DatabaseConnection per la gestione del pool di connessioni.
- **DAO Pattern:** Per disaccoppiare la logica di accesso ai dati.
- **MVC:** Architettura portante del client JavaFX.
- **DTO (Message Pattern):** Uso di Request/Response per la serializzazione dei dati sul canale TCP.

JavaDoc

Il progetto implementa una documentazione capillare seguendo gli standard Oracle. Ogni metodo rilevante include tag specifici (@author, @param, @return, @throws) per descriverne il comportamento e le eccezioni gestite.

Generazione documentazione:

```
mvn javadoc:javadoc
```

Bibliografia

- * Oracle, *How to Write Doc Comments for the Javadoc Tool*.
- * PostgreSQL Documentation.
- * Apache Maven Project.
- * HikariCP Project.
- * JavaFX Documentation.
- * Wikipedia - Data Access Object